QUANTITATIVE RISK MANAGEMENT IN CREDIT PORTFOLIOS: PD MODEL SELECTION AND ITS EFFECT ON VAR AND ES

Written By: Kyle Shi, Vito Zhao

AUGUST 11, 2025

UNIVERSITY OF WATERLOO ACTSC 445 SUMMER 2025

Contents

Abstract

This study examines how the choice of probability of default (here in the later of this report, we will refer it as PD) model influences portfolio credit risk measures within a quantitative risk management (QRM) framework. The dataset we use is called "Give Me Some Credit" and we evaluate four models: regularized logistic regression, XGBoost, CatBoost, and Random Forest. Model performance is assessed using ROC-AUC, PR-AUC, and the Kolmogorov-Smirnov (KS) statistics, with KS-optimal thresholds selected on validation data. The PD estimates are then embedded into a Monte Carlo of independent Bernoulli defaults to generate portfolio loss distributions. Assuming fixed exposure at default (EAD) and fixed loss given default (LGD), we computed VaR and ES at 95%, 97.5%, and 99%. The results show that Random Forest produces the lowest VaR/ES and L1-logistic shows the highest under independent defaults.

1. Introduction

Probability-of-default (PD) models are a core tool in credit risk management because they feed into pricing, underwriting, provision estimates, stress testing, and capital planning. Traditional logistic-regression scorecards remain common for their simplicity and transparency. However, classic scorecards have several drawbacks. The log-odds linearity and additivity assumptions make it hard to capture complex non-linear effects or higher-order interactions unless they are hand-engineered; widespread use of binning and monotone constraints aids interpretability but can smooth away local signal and make results sensitive to design choice[1,2].

Modern machine-learning (ML) methods—such as XGBoosting, random forests, and CatBoost—often model PD better than logistic scorecards: they capture non-linearities and interactions without hand-crafted bins, handle mixed and imbalanced data with built-in regularization, and, when paired with post-hoc calibration, produce more accurate probabilities as well as higher discrimination[3,4].

When evaluating PD models, it is important to separate discrimination from portfolio risk. Discrimination measures how well a model ranks defaulters ahead of non-defaulters. Throughout, we evaluate discrimination with ROC-AUC, PR-AUC, and the Kolmogorov-Smirnov (KS) statistics. ROC-AUC is the probability that a randomly chosen defaulter receives a higher score than a randomly chosen non-defaulter (0.5 is random, 1.0 is perfect). PR-AUC is the area under the precision-recall curve, which emphasizes the minority (default) class and equals the default rate for a random classifier. KS is the maximum separation $\max_t\{TPR(t) - FPR(t)\}$. TPR is the True Positive Rate which is the fraction of actual defaulters correctly

flagged at a given threshold; and FPR is the False Positive Rate which is the fraction of non-defaulters incorrectly flagged as defaulters. While these metrics capture ranking quality, they do not by themselves reveal how PD choice affects loss tails and required capital.

This paper closes that gap by linking PD estimates to portfolio loss distributions and tail-risk measures including Value-at-Risk (VaR) and Expected Shortfall (ES). Using a simple, reproducible simulation, we translate each model's PDs into portfolio losses and compare the induced VaR/ES across a sparse logistic baseline and three ML alternatives. We show that improvements in discrimination from ML models are accompanied by meaningfully lower VaR and ES under common assumptions, clarifying the risk impact beyond rank statistics.

The rest of the paper proceeds as follows. Section 2 reviews the model. Section 3 describes the dataset and preprocessing. Section 4 details the modeling and evaluation methodology. Section 5 reports discrimination results and the corresponding VaR/ES. Section 6 discusses implications and extensions.

2. Background for models

Below are the models used in this study; for each we give a brief background:

- **L1-regularized logistic regression (baseline)**: Logistic regression treats default as a 0/1 outcome and maps a weighted sum of features through an S-shaped curve to produce a probability. The L1 (lasso) penalty pushes unhelpful coefficients toward zero, acting as automatic feature selection and limiting overfitting. It is highly interpretable and often well-calibrated, but its linear, additive form misses nonlinear patterns and interactions unless engineered.
- **Gradient boosting (XGBoost)**: Boosting builds trees sequentially, each new tree correcting what the current ensemble gets wrong. XGBoost strengthens this with learning-rate shrinkage, regularization, and subsampling, delivering strong accuracy on tabular credit data while controlling overfitting. It discovers complex interactions automatically, but performance is sensitive to hyperparameters; early stopping and cross-validation are important, and probability calibration is usually advisable for PD work.
- **CatBoost:** CatBoost is a boosting algorithm designed to handle categorical features well. It uses "ordered" target statistics and ordered boosting to reduce target leakage and overfitting, so it often performs well out-of-the-box on real credit data with mixed types. It learns nonlinearities and interactions with minimal tuning; like other ensembles, its raw scores benefit from simple post-hoc probability calibration.

- **Random forest**: A random forest averages many decision trees grown on bootstrap samples and random feature subsets, so their errors tend to cancel. It naturally captures nonlinearities and interactions, is robust to noisy features and moderate class imbalance, and needs little tuning. On the downside, its explanations are indirect, and raw probabilities can be poorly calibrated, so a light calibration step helps.

3. Data

3.1 Dataset Description

Our study uses the publicly available "Give Me Some Credit" dataset from Kaggle, which contains anonymized borrower information for predicting the likelihood of experiencing a serious financial delinquency within the next two years. The dataset includes 150,000 observations with the binary target variable SeriousDlqin2yrs (1 = default, 0 = non-default). The following is an explanation of each variable being used in the datasets provided by Kaggle.

Table. 1. Data Dictionary

Variable Name	**Description**	**Type**
SeriousDlqin2yrs	Person experienced 90 days past due delinquency or worse	Binary
RevolvingUtilizationOfUnsecuredLines	Total balance on credit cards and personal lines of credit except real estate and no installment debt like car loans divided by the sum of credit limits	Percentage
age	Age of borrower in years	Integer
NumberOfTime30-59DaysPastDueNotWorse	Number of times borrowers have been 30-59 days past due but no worse in the last 2 years.	Integer
DebtRatio	Monthly debt payments, alimony,living costs divided by monthly gross income	Percentage
MonthlyIncome	Monthly income	Real
NumberOfOpenCreditLinesAndLoans	Number of Open loans (installment like car loan or mortgage) and Lines of credit (e.g. credit cards)	Integer
NumberOfTimes90DaysLate	Number of times borrower has been 90 days or more past due.	Integer

NumberRealEstateLoansOrLines	Number of mortgage and real estate loans including home equity lines of credit	Integer
NumberOfTime60-89DaysPastDueNotWorse	Number of borrowers has been 60-89 days past due but no worse in the last 2 years.	Integer
NumberOfDependents	Number of dependents in family excluding themselves (spouse, children etc.)	Integer

3.2 Data Preprocessing

To ensure quality and consistency in PD estimation, we implemented a structured data cleaning and feature engineering process:

1. Initial Cleaning
   - Removed unnamed index column.
   - Filtered the dataset to include borrowers only from aged 20 to 100 years
2. Income Processing
   - Created NoIncomeFlag for borrowers reporting MonthlyIncome = 0 (to capture cases prone to extreme DebtRatio)
   - Grouped borrowers into age brackets (20-29, 30-39, …, 90-100) and computed median income for each group. For missing MonthlyIncome, we use corresponding age group median income as an estimate.
   - Created IncomeImputedFlag to track estimated values
3. Debt Ratio Capping
   - Ensure all DebtRatio is numeric and replace any infinite values with NaN
   - Applied realistic caps based on income status (where 0 and NA income results in extreme high DebtRatio):
     a) 0-Income borrowers and Positive-income with no imputed values: cap at 10.0
     b) Positive-income with imputed values: cap at 2.0
   - Fill any remaining NaNs with the median of observed values
4. Dependents Processing
   - Created DependentsMissingFlag for missing NumberOfDependents
   - Created LargeFamilyFlag for families with greater than or equal to 5 dependents
   - Top-coded extreme dependent counts at 5 (as we see certain numbers going more than 5 which is not very realistic)

- Imputed missing values with the median of the top-coded distribution

5. Past-Due Variable Processing
   - For each of the three delinquency count variables:
     a) Converted to numeric and replaced special codes 96 and 98 with NaN tracking them with *_MissingFlag variables
     b) Imputed missing values with median from valid cases
     c) Capped extreme counts at 10
6. Feature and Target Separation
   - Defined SeriousDlqin2yrs as the target variable.
   - Retained clean and engineered features for modeling
7. Data Splitting and Reproducibility
   - Stratified Train (60%) /Validation (20%) / Test (20%) split using a fixed random seed
   - Saved split indices to ensure identical subsets across runs

3.3 Rationale for Data Adjustment

The preprocessing modifications were designed based on our knowledge to improve data quality, preserve predictive signals, and prevent distortions during model training. Filtering out ages below 20 and above 100 removes improbable profiles which improve generalizability. Age group medians for missing income produces contextually realistic replacements, avoiding bias introduced by global averages. Capping extreme ratios such as DebtRatio and NumberofDependets mitigates the influence of implausible values and ensure that no scaling issue will affect our model results. Introducing category flags retains potential predictive signals embedded in missingness patterns. Overall, these steps align the dataset with realistic credit risk expectations and reduce noise which creates stable inputs for PD estimation and subsequent QRM analysis.

4. Methodology

We enforce strict determinism: a single global seed (445), single threaded numeric backend, and fixed PYTHONHASHSEED. Train/validation/test indices are created once via stratified splits (60/20/20) and persisted to disk, so every model is trained and evaluated on identical rows across runs to make sure reproducibility.

Hyperparameters are chosen on the training subset via 3-fold stratified cross-validation using ROC-AUC as the scoring metric. The selected configuration is then refit on Train and Validation then evaluated on the untouched Test subset to avoid test leakage.

All four models are trained as scikit-learn pipelines (except CatBoost which is trained directly); preprocessing inside the pipeline is minimal model-specific:

- **L1-Logit:**
  Pipeline: StandardScaler → LogisticRegression(penalty="l1", solver="saga", max_iter=5000, class_weight="balanced")
  Tuned: C ∈ $[10^{-3}, 10]$ (16 log-spaced)
- **XGBoost (hist):**
  Pipeline: SimpleImputer(median) → XGBClassifier(objective="binary:logistic", eval_metric="auc", tree_method="hist", subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0, reg_alpha=0.0, scale_pos_weight=#neg/#pos)
  Tuned: n_estimators ∈ [400, 800], max_depth ∈ [4, 6], learning_rate [0.05, 0.1], min_child_weight ∈ [1, 3] (inputs cast to float32)
- **CatBoost (CPU):**
  CatBoostClassifier(iterations=2000, early_stopping_rounds=100, loss_function="Logloss", eval_metric="AUC", class_weights=[1, #neg/#pos])
  Tuned: depth ∈ [6, 8], learning_rate ∈ [0.05, 0.1], l2_leaf_reg=3
- **Random Forest:**
  Pipeline: SimpleImputer(median) → RandomForestClassifier(class_weight="balanced_subsample")
  Tuned: n_estimators ∈ [400, 800], max_depth ∈ [None, 12], min_samples_leaf ∈ [1, 5], max_features ∈ ["sqrt", 0.5]

We report ROC-AUC (discrimination), PR-AUC (average precision, informative under imbalance), and KS, defined as $\max_t\{TPR(t) - FPR(t)\}$. Operational thresholds are set once per model by maximizing KS on the Validation set, and the same threshold is then applied on the Test set to compute confusion matrices and classification report. We also present combined ROC, PR, and KS-vs-threshold plots on the test set and print a model summary table including ROC-AUC, PR_AUC, KS, and KS-threshold.

Lastly, we translate Test-set PDs into portfolio losses via a simple Monte Carlo with independent defaults. For account i, default occurs with probability p_i; loss is W_i if default and 0 otherwise, with $W_i = EAD_i \times LGD_i$. EAD (exposure at default) is the outstanding exposure at default and LGD (loss given default) is the fraction of exposure not recovered (1 − recovery rate). In our baseline we set $EAD_i = 1$ and $LGD_i = 1$, so losses are measured in default counts; monetary VaR/ES follow by inserting actual EAD/LGD. For each model we run 8000 i.i.d. Bernoulli trials with probability p_i and sum portfolio loss; RNG seeds are derived deterministically from the global seed and more name fore reproducibility. From simulated

losses we computed Expected Loss, VaRα and ESα at $\alpha \in \{0.95, 0.975, 0.99\}$. VaR uses a conservative empirical quantile ("higher" rule); ES is the mean of the tail L≥VaR. We also present VaR and ES as rates (loss ÷ total exposure) to normalize for portfolio size and EAD/LGD scale. This makes model and scenario comparisons apples-to-apples across time and samples, and it preserves confidentiality of absolute exposures.

5. Results

5.1 Discrimination: ROC-AUC and PR-AUC

For ROC, the closer the curve is to the top-left corner the better; for PR, the closer to the top-right the better. AUC summarizes these shapes into a single score.

In Figures 1–2, across four models, XGBoost delivered the best overall discrimination, with ROC-AUC = 0.872 and PR-AUC = 0.410, followed closely by Random Forest (ROC-AUC = 0.863, PR-AUC = 0.392) and CatBoost (ROC-AUC = 0.862, PR-AUC = 0.392). The L1-Logit baseline trailed as expected (ROC-AUC = 0.830, PR-AUC = 0.365). Because the test set is imbalanced, we emphasize PR alongside ROC; all models achieve PR-AUC well above the no-skill baseline implied by the test-set prevalence, indicating substantial lift.

In an imbalanced setting, PR is often more informative than ROC because it emphasizes the precision and recall tradeoff, so consistent gains on both curves signal a robust improvement rather than a metric artifact. In the low false positive rate region that screening cares about, higher curves mean more defaulters captured for the same false alarm budget. Curves can also cross, so judge performance in the intended operating range.

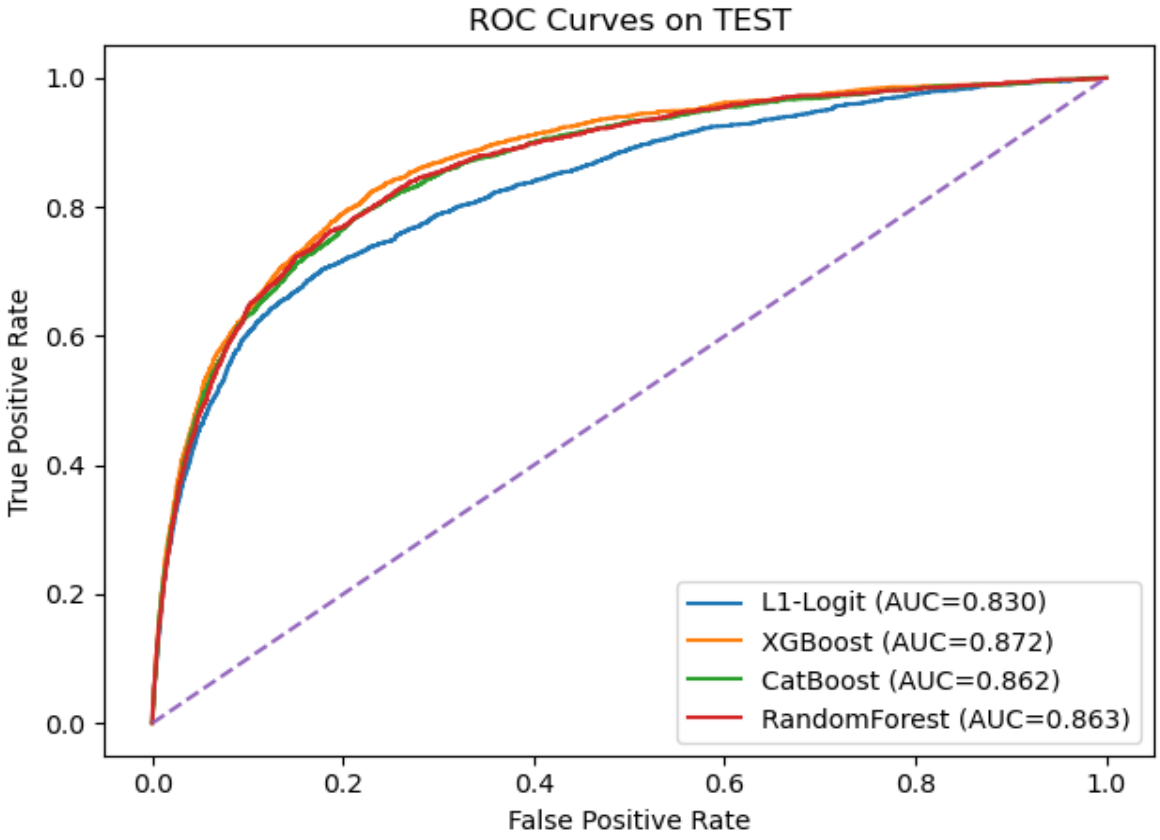

Fig. 1. Test-Set ROC Curves by Model

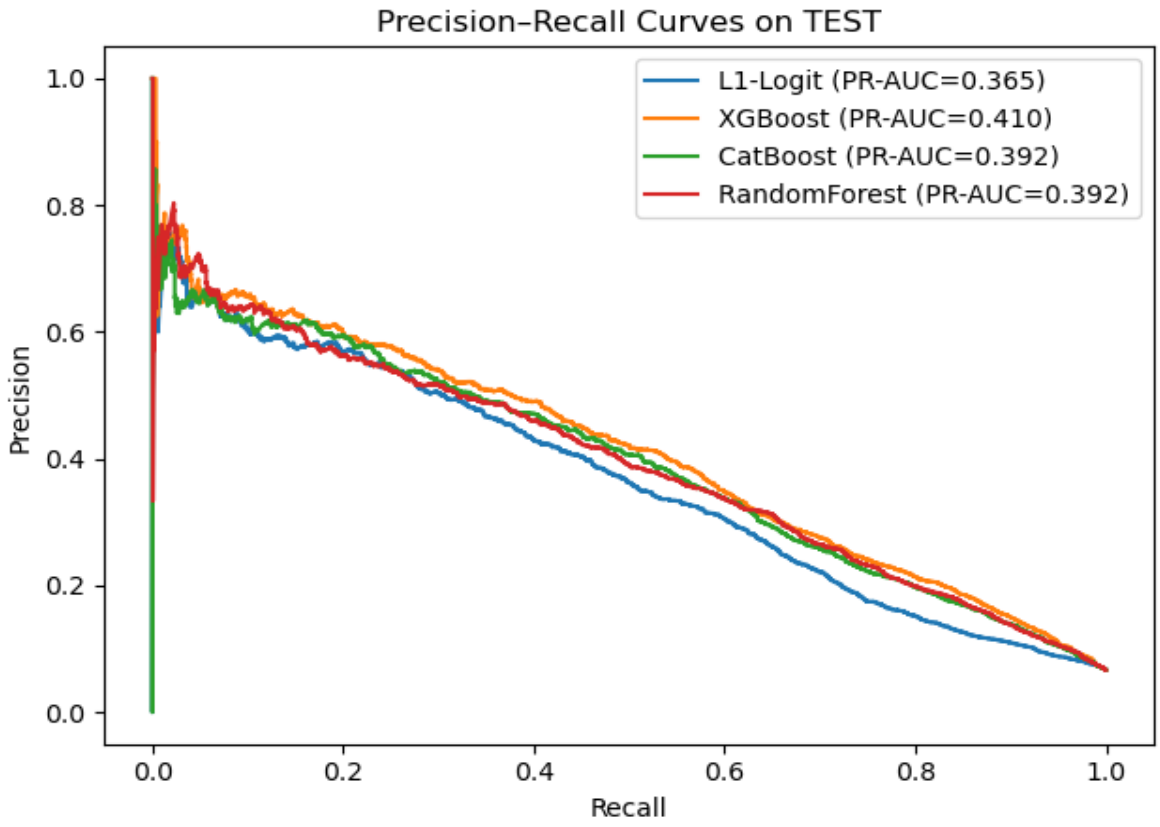

Fig. 2. Precision–Recall Curves on the Test Set

5.2 Operating point: KS curves and trade-offs

We set each model's operating point by maximizing KS on the validation set and reused the same threshold on the test set. On the test set, peak KS values cluster around 0.59 (XGBoost), 0.58 (Random Forest), 0.57 (CatBoost), and 0.53 (L1-Logit), with corresponding thresholds in the 0.40–0.46 range (Fig. 3).

The KS curve plots TPR − FPR as the decision threshold varies; the peak marks the threshold that maximizes separation between defaulters and non-defaulters. In Fig. 3, the boosted/tree ensembles show a higher, broader apex than L1-Logit, indicating stronger separation and a wider plateau of near-optimal thresholds which is useful when operating policies or class balance shift over time.

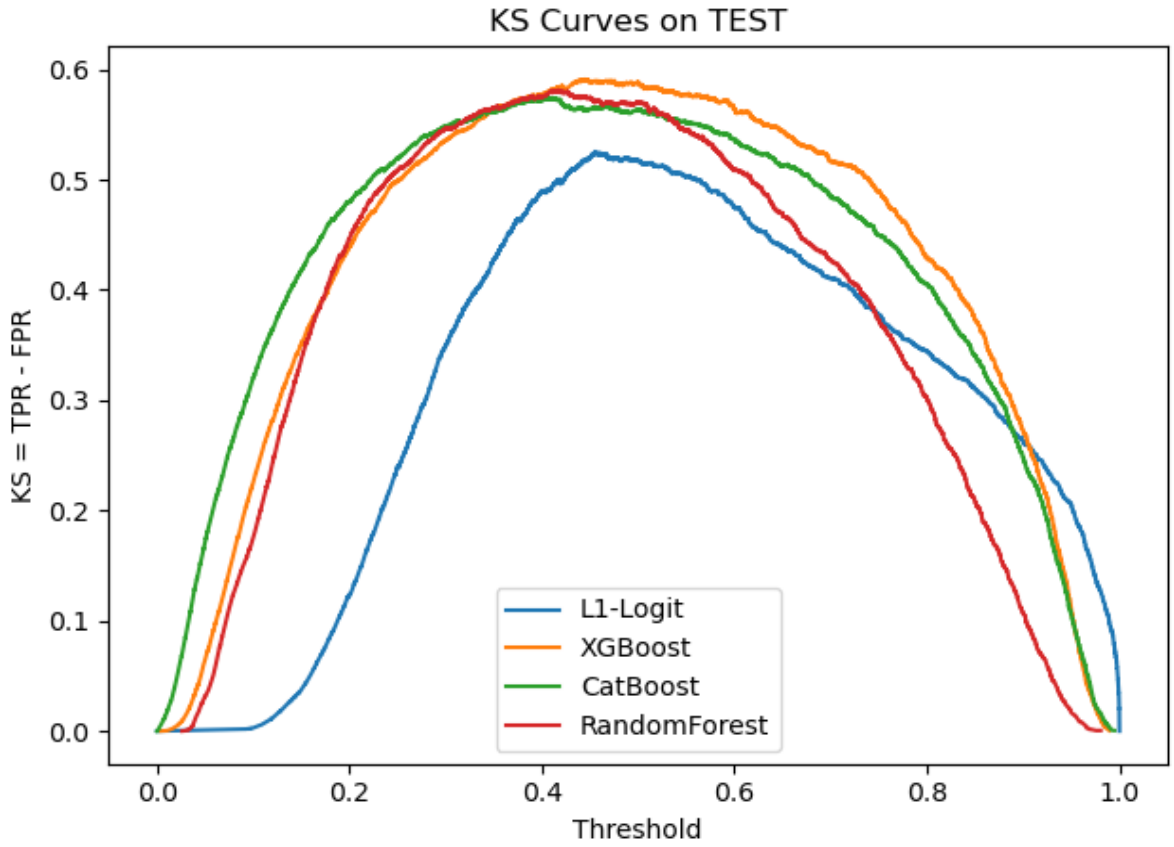

Fig. 3. KS Curves on the Test Set

At the KS optimal operating point (chosen on validation and held fixed on test), all models favor higher recall than precision, because the KS statistic (TPR − FPR) rewards capturing defaulters. In credit risk a missed defaulter costs more than an extra review, so this tilt is desirable. L1-Logit attains the highest precision but the lowest recall. XGBoost delivers the strongest recall; Random Forest follows closely, while CatBoost trades some recall for

comparatively higher precision among the ensembles. Because KS is rank based, post hoc calibration may shift the numeric threshold without changing the ordering of models.

The selected operating point should be rechecked whenever PDs are recalibrated or the class prevalence changes and supplemented with recall at fixed FPR or precision within the top-k percent. At the KS optimal threshold selected on validation and held fixed on test, Table 2 reports precision and recall at the KS-optimal threshold; full confusion matrices and additional metrics (FPR, KS, and acceptance rate) appear in Appendix.

Table. 2. Precision and Recall at the KS-optimal threshold (Test Set)

Model	Precision	Recall
L1-Logit	0.255	0.657
XGBoost	0.206	0.813
CatBoost	0.241	0.726
Random Forest	0.226	0.763

5.3 Portfolio translation: VaR and ES

Using the simulation described in the Methodology (independent defaults; EAD = 1, LGD = 1), we translate test-set PDs into portfolio loss distributions and report mean loss rates and tail measures. Table 3 reports VaR and ES at 95%, 97.5%, and 99% as loss rates (loss divided by total exposure), not dollar amounts. Relative to L1-Logit, all ML models materially reduce tail risk; at the 99% level, VaR and ES fall by about 16% for XGBoost, 26% for CatBoost, and 53% for Random Forest, and the same ordering holds at 95% and 97.5%. Complete numerical results, including the original per-model VaR and ES values, are provided in Appendix. To interpret the levels, note a mechanical feature of the independent-defaults benchmark.

Under independent defaults and a large portfolio, loss variance scales as $p(1-p)/N$, so tail quantiles lie very close to the mean. The key comparison is therefore relative VaR/ES across models rather than the gap between VaR and the mean.

Notably, tail-risk ordering does not mirror discrimination perfectly: Random Forest yields the lowest VaR/ES despite slightly lower AUC than XGBoost/CatBoost. This reflects that portfolio loss tails depend on the full PD distribution and calibration, not just ranking models that push probability mass away from the mid-range and calibrate the upper tail can produce tighter loss tails.

Table. 3. Portfolio Loss (Rates): Mean, VaR, and ES at 0.95, 0.975 and 0.99 (Test Set)

Model	Mean Loss Rate	$VaR_{0.95}$ (Rate)	$ES_{0.95}$ (Rate)	$VaR_{0.975}$ (Rate)	$ES_{0.975}$ (Rate)	$VaR_{0.99}$ (Rate)	$ES_{0.99}$ (Rate)
L1-Logit	0.374	0.378	0.379	0.379	0.379	0.379	0.380
XGBoost	0.315	0.319	0.319	0.319	0.320	0.320	0.321
CatBoost	0.276	0.279	0.280	0.280	0.281	0.281	0.281
Random Forest	0.175	0.178	0.179	0.178	0.179	0.179	0.180

5.4 Practical Implications

Overall, the machine-learning models (XGBoost, Random Forest, CatBoost) outperform the logistic baseline on discrimination (ROC-AUC, PR-AUC, KS) and, crucially, this ranking advantage carries through to portfolio risk: when test-set PDs are mapped to losses, ML models produce materially lower VaR and ES than logistic, with the ordering stable across confidence levels. Using the KS-selected operating point, ML models also achieve higher recall of defaulters, reducing missed-default risk for screening. In practical terms, these gains imply lower capital and stress-loss buffers for the same risk appetite, more efficient limit setting, and steadier collections planning; absolute levels depend on PD calibration and EAD/LGD assumptions, but the relative pattern is robust.

6. Conclusion

Our project asked a practical QRM question: does the choice of PD model have a material impact on portfolio tail risk? Using a train/validate/test workflow with KS-selected operating thresholds and fixed seeds for determinism, we found that modern ML models beat the L1-logit baseline on discrimination and that this advantage carries through to VaR and ES. In our results, XGBoost delivered the strongest discrimination (with CatBoost and Random Forest close behind), while logistic trailed, consistent with the ROC and PR curves reported in the Results section. Mapping test-set PDs into a simple MC with independent defaults (EAD = LGD = 1), all three ML models reduced tail risk relative to logistic. Random Forest achieved the lowest VaR and ES. These findings indicate that better discrimination translated into tighter loss tails under common assumptions, implying lower capital buffers for the same risk appetite.

Practically, this means PD model selection changes tail metrics that drive planning and capital. Across standard tail levels, the ordering of models is consistent, so the same portfolio can exhibit meaningfully different VaR or ES depending on the PD engine. For institutions operating with fixed risk appetites, stronger discrimination allows companies to hold less

capital for the same target percentile and expand exposure while keeping tail risk constant.

In future work, we will model default dependence rather than independence by rerunning Monte Carlo with a one-factor Gaussian to inject correlated defaults and see how VaR or ES change. We will also replace EAG=LGD=1 with realistic distributions and allow wrong-way risk so LGD worsens when defaults spike. Instead of picking thresholds purely by KS, we will optimize by capital impact and report the shift in VaR and ES. Finally, we will add uncertainty bands using bootstrap Cis and stabilize simulations with common random numbers. With these refinements, our tail estimates can be more decision-ready and robust.

Overall, the conclusion is affirmative. Under our design, ML models deliver narrower loss tails than a regularized logistic baseline. For risk managers, this translates into tangible levers for capital efficiency, pricing, and limit setting, provided the models are governed with proper validation, periodic drift monitoring, and sensitivity checks to default dependence and loss-given-default assumptions.

Reference

[1] Montevechi, A. A., de Carvalho Miranda, R., Medeiros, A. L., & Montevechi, J. A. B. (2024). Advancing credit risk modelling with Machine Learning: A comprehensive review of the state-of-the-art. Engineering Applications of Artificial Intelligence, 137, 109082. doi:10.1016/j.engappai.2024.109082.

[2] Crouhy, M., Galai, D., & Mark, R. (2000). A comparative analysis of current credit risk models. *Journal of Banking & Finance, 24*(1), 59–117.

[3] Vidovic, L., & Yue, L. (2020). *Machine learning and credit risk modelling* (White paper). S&P Global Market Intelligence.

[4] Galindo, J., & Tamayo, P. (2000). Credit risk assessment using statistical and machine learning: Basic methodology and risk modeling applications. *Computational Economics, 15*(1–2), 107–143.

Appendix

See Project code output and Project code below.

All working file can also be found here: https://github.com/KM-MengXun/ML-Credit-Risk

```
Fitting 3 folds for each of 16 candidates, totalling 48 fits
Best C: 0.011659144011798317
CV ROC-AUC (mean): 0.8169151077959099
Validation ROC-AUC: 0.8184
Validation PR-AUC : 0.3630
Validation KS     : 0.5006 @ threshold 0.4859

Validation Confusion Matrix:
[[23954  4038]
 [  712  1293]]

Validation Classification Report:
              precision    recall  f1-score   support

           0     0.9711    0.8557    0.9098     27992
           1     0.2425    0.6449    0.3525      2005

    accuracy                         0.8417     29997
   macro avg     0.6068    0.7503    0.6312     29997
weighted avg     0.9224    0.8417    0.8725     29997


=== TEST RESULTS (using validation KS threshold) ===
Test ROC-AUC: 0.8303
Test PR-AUC : 0.3653
Test KS     : 0.5255

Test Confusion Matrix:
[[24149  3844]
 [  688  1317]]

Test Classification Report:
              precision    recall  f1-score   support

           0     0.9723    0.8627    0.9142     27993
           1     0.2552    0.6569    0.3676      2005

    accuracy                         0.8489     29998
   macro avg     0.6137    0.7598    0.6409     29998
weighted avg     0.9244    0.8489    0.8777     29998

Fitting 3 folds for each of 16 candidates, totalling 48 fits
Best params: {'xgb__learning_rate': 0.05, 'xgb__max_depth': 4, 'xgb__min_child_weight': 3.0, 'xgb__n_estimators': 400}
CV ROC-AUC : 0.8576803590955926
```

```
Validation ROC-AUC: 0.8695
Validation PR-AUC : 0.4082
Validation KS     : 0.5864 @ threshold 0.4632

Validation Confusion Matrix:
[[21692  6300]
 [  378  1627]]

Validation Classification Report:
              precision    recall  f1-score   support

           0     0.9829    0.7749    0.8666     27992
           1     0.2052    0.8115    0.3276      2005

    accuracy                         0.7774     29997
   macro avg     0.5941    0.7932    0.5971     29997
weighted avg     0.9309    0.7774    0.8306     29997


=== TEST RESULTS (using validation KS threshold) ===
Test ROC-AUC: 0.8717
Test PR-AUC : 0.4100
Test KS     : 0.5942

Test Confusion Matrix:
[[21742  6251]
 [  375  1630]]

Test Classification Report:
              precision    recall  f1-score   support

           0     0.9830    0.7767    0.8678     27993
           1     0.2068    0.8130    0.3298      2005

    accuracy                         0.7791     29998
   macro avg     0.5949    0.7948    0.5988     29998
weighted avg     0.9312    0.7791    0.8318     29998

Best CatBoost params: {'depth': 6, 'learning_rate': 0.05, 'l2_leaf_reg': 3.0}
Validation ROC-AUC: 0.8717359662432942
Validation PR-AUC: 0.4126
Validation KS    : 0.5899 @ threshold 0.4953

Validation Confusion Matrix:
```

```
[[22153  5839]
 [  404  1601]]

Validation Classification Report:
              precision    recall  f1-score   support

           0     0.9821    0.7914    0.8765     27992
           1     0.2152    0.7985    0.3390      2005

    accuracy                         0.7919     29997
   macro avg     0.5986    0.7950    0.6078     29997
weighted avg     0.9308    0.7919    0.8406     29997


=== TEST RESULTS (CatBoost, using validation KS threshold) ===
Test ROC-AUC: 0.8618
Test PR-AUC : 0.3919
Test KS     : 0.5715

Test Confusion Matrix:
[[23404  4589]
 [  548  1457]]

Test Classification Report:
              precision    recall  f1-score   support

           0     0.9771    0.8361    0.9011     27993
           1     0.2410    0.7267    0.3619      2005

    accuracy                         0.8288     29998
   macro avg     0.6091    0.7814    0.6315     29998
weighted avg     0.9279    0.8288    0.8651     29998

Fitting 3 folds for each of 16 candidates, totalling 48 fits
Best RF params: {'rf__max_depth': None, 'rf__max_features': 'sqrt', 'rf__min_samples_leaf': 5, 'rf__n_estimators': 800}
CV ROC-AUC    : 0.853579968341507
Validation ROC-AUC: 0.8654
Validation PR-AUC : 0.3950
Validation KS     : 0.5811 @ threshold 0.2539

Validation Confusion Matrix:
[[22703  5289]
 [  461  1544]]
```

```
Validation Classification Report:
              precision    recall  f1-score   support

           0     0.9801    0.8111    0.8876     27992
           1     0.2260    0.7701    0.3494      2005

    accuracy                         0.8083     29997
   macro avg     0.6030    0.7906    0.6185     29997
weighted avg     0.9297    0.8083    0.8516     29997


=== TEST RESULTS (Random Forest, using validation KS threshold) ===
Test ROC-AUC: 0.8634
Test PR-AUC : 0.3923
Test KS     : 0.5761

Test Confusion Matrix:
[[22746  5247]
 [  476  1529]]

Test Classification Report:
              precision    recall  f1-score   support

           0     0.9795    0.8126    0.8883     27993
           1     0.2256    0.7626    0.3483      2005

    accuracy                         0.8092     29998
   macro avg     0.6026    0.7876    0.6183     29998
weighted avg     0.9291    0.8092    0.8522     29998


=== TEST summary ===
       Model  ROC-AUC   PR-AUC       KS  KS_threshold
     XGBoost 0.871716 0.409956 0.594207      0.455568
RandomForest 0.863386 0.392263 0.576084      0.255425
    CatBoost 0.861847 0.391867 0.571462      0.407810
    L1-Logit 0.830288 0.365291 0.525482      0.455505

=== TEST Portfolio VaR & ES (counts and rates; assumes indep. defaults, EAD=1, LGD=1) ===
       Model     MeanLoss   StdLoss  VaR@95        ES@95  VaR@97.5      ES@97.5  VaR@99        ES@99  MeanLossRate  VaRrate@95  ESrate@95
VaRrate@99  ESrate@99
    L1-Logit 11203.885625 77.528844 11331.0 11362.782716   11357.0 11384.113300 11382.0 11408.518072      0.373488    0.377725   0.378785
0.379425   0.380309
     XGBoost  9444.348375 66.080618  9555.0  9581.977556    9575.0  9600.158416  9598.0  9622.225000      0.314833    0.318521   0.319421
```

```
0.319955   0.320762
    CatBoost  8274.475625 63.225694  8378.0  8403.397500    8396.0  8419.846535  8421.0  8439.150000      0.275834    0.279285   0.280132
0.280719   0.281324
RandomForest  5244.216250 55.603381  5336.0  5359.689487    5354.0  5376.450495  5374.0  5395.423529      0.174819    0.177879   0.178668
0.179145   0.179859

Saved PDF: run_output.pdf
```

```
# ============================ Export terminal output to pdf ============================
import os, random, gc
import sys, io, atexit
class Tee:
    def __init__(self, *files): self.files = files
    def write(self, x):
        for f in self.files: f.write(x)
    def flush(self):
        for f in self.files: f.flush()

_log = open("run_output.txt", "w", encoding="utf-8")
sys.stdout = Tee(sys.stdout, _log)   # print() goes to console + file
sys.stderr = Tee(sys.stderr, _log)   # errors too

@atexit.register
def _close_log():
    try: _log.close()
    except: pass


# ============================ DETERMINISM HEADER ============================
import os, random, gc

SEED = 445                           # single source of truth
DETERMINISTIC = True                 # flip to False for speed over strict repeatability
THREADS = 1 if DETERMINISTIC else max(1, os.cpu_count() // 2)

# Make numeric libs single-threaded to avoid nondeterministic reductions
os.environ["OMP_NUM_THREADS"]        = "1" if DETERMINISTIC else str(THREADS)
os.environ["OPENBLAS_NUM_THREADS"]   = "1" if DETERMINISTIC else str(THREADS)
os.environ["MKL_NUM_THREADS"]        = "1" if DETERMINISTIC else str(THREADS)
os.environ["VECLIB_MAXIMUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
os.environ["NUMEXPR_NUM_THREADS"]    = "1" if DETERMINISTIC else str(THREADS)

# Python & NumPy RNG
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
# ========================================================================

# Core libs
import pandas as pd
import numpy as np
from pathlib import Path

```

```
# Sklearn base imports
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    roc_auc_score, roc_curve, precision_recall_curve, average_precision_score,
    confusion_matrix, classification_report
)

# Model libraries
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
from sklearn.ensemble import RandomForestClassifier

# Plotting
import matplotlib.pyplot as plt

# For reproducibility of random numbers in simulations
import hashlib

# ======================================== Load Data & Data Pre-processing ========================================
# --- file paths (edit TEST_PATH if needed) ---
#   r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv"
#   r"D:\Github\ML-Credit-Risk\Dataset\GiveMeSomeCredit\cs-training.csv"

df = pd.read_csv(r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv")

# drop index column if exists
df = df.drop(columns=["Unnamed: 0"], errors="ignore")

# keep ages 20–100
df = df[(df["age"] >= 20) & (df["age"] <= 100)].copy()

# MonthlyIncome: keep 0s, impute only NAs by age-group median ===
df["NoIncomeFlag"] = (df["MonthlyIncome"] == 0).astype("int8")

bins   = [20, 30, 40, 50, 60, 70, 80, 90, 100]
labels = ["20-29", "30-39", "40-49", "50-59", "60-69", "70-79", "80-89", "90-100"]
df["age_group"] = pd.cut(df["age"], bins=bins, labels=labels, right=False)

median_by_group = (
    df.loc[df["MonthlyIncome"] > 0]
```

```
   89 |       .groupby("age_group", observed=False)["MonthlyIncome"]
   90 |       .median()
   91 | )
   92 | mi_na_mask = df["MonthlyIncome"].isna()
   93 | df.loc[mi_na_mask, "MonthlyIncome"] = df.loc[mi_na_mask, "age_group"].map(median_by_group)
   94 | df["IncomeImputedFlag"] = mi_na_mask.astype("int8")
   95 | df = df.drop(columns=["age_group"])
   96 |
   97 | # DebtRatio: coerce, remove inf, and cap per new rules (0-income=10, imputed=2, normal=10) ===
   98 | dr = pd.to_numeric(df["DebtRatio"], errors="coerce")
   99 | dr = dr.mask(np.isinf(dr), np.nan)  # remove ±inf safely
  100 |
  101 | # Masks from existing columns/flags
  102 | pos_income    = df["MonthlyIncome"] > 0
  103 | was_imputed   = df["IncomeImputedFlag"].eq(1)  # MonthlyIncome was NA and then imputed by median
  104 | zero_income   = df["NoIncomeFlag"].eq(1)       # MonthlyIncome == 0 (flag set before imputation)
  105 | normal_income = pos_income & (~was_imputed)    # positive and not imputed
  106 |
  107 | # Caps
  108 | REALISTIC_CAP = 10.0
  109 | IMPUTED_CAP   = 2.0
  110 |
  111 | # Start from a clean series
  112 | dr_series = pd.Series(dr, index=df.index)
  113 |
  114 | # Apply caps with lower bound at 0 (avoid negatives)
  115 | dr_series.loc[was_imputed]   = np.clip(dr_series.loc[was_imputed],   0.0, IMPUTED_CAP)
  116 | dr_series.loc[zero_income]   = np.clip(dr_series.loc[zero_income],   0.0, REALISTIC_CAP)
  117 | dr_series.loc[normal_income] = np.clip(dr_series.loc[normal_income], 0.0, REALISTIC_CAP)
  118 |
  119 | # Fill any remaining NaNs with the median of observed values
  120 | dr_series = dr_series.fillna(np.nanmedian(dr_series.values))
  121 | df["DebtRatio"] = dr_series.astype(float)
  122 |
  123 | # NumberOfDependents: flag missing, top-code, impute median, large-family flag
  124 | dep_na_mask = df["NumberOfDependents"].isna()
  125 | df["DependentsMissingFlag"] = dep_na_mask.astype("int8")
  126 |
  127 | # top-code extremes at 5 before imputation
  128 | dep_series = df["NumberOfDependents"].copy()
  129 | dep_series = dep_series.where(dep_series.isna() | (dep_series <= 5), 5)
  130 | dep_median = int(dep_series.median(skipna=True))
  131 | dep_series = dep_series.fillna(dep_median).astype("int8")
  132 |
```

```
df["NumberOfDependents"] = dep_series
df["LargeFamilyFlag"] = (df["NumberOfDependents"] >= 5).astype("int8")

# Number of Times Past Due Clean up
cols_past_due = [
    "NumberOfTime30-59DaysPastDueNotWorse",
    "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"
]
for col in cols_past_due:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df[f"{col}_MissingFlag"] = df[col].isin([96, 98]).astype(int)
    df[col] = df[col].replace({96: np.nan, 98: np.nan})
    median_val = df[col].median(skipna=True)
    df[col] = df[col].fillna(median_val)
    df[col] = np.where(df[col] > 10, 10, df[col]).astype(int)

# ======================================== Split the dataset ========================================
TARGET = "SeriousDlqin2yrs"
X = df.drop(columns=[TARGET])
y = df[TARGET].astype(int)

# First split into train+val and test (stratified)
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED
)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.25, stratify=y_trainval, random_state=SEED
)

# ---------------- Persist splits so future runs use identical rows --------------
split_dir = Path("./_splits"); split_dir.mkdir(exist_ok=True)
f_train = split_dir / "train_idx.npy"
f_val   = split_dir / "val_idx.npy"
f_test  = split_dir / "test_idx.npy"

if all(f.exists() for f in (f_train, f_val, f_test)):
    # Rebuild splits from saved indices
    idx_train = np.load(f_train, allow_pickle=False)
    idx_val   = np.load(f_val,   allow_pickle=False)
    idx_test  = np.load(f_test,  allow_pickle=False)
    X_train, y_train = X.loc[idx_train], y.loc[idx_train]
    X_val,   y_val   = X.loc[idx_val],   y.loc[idx_val]
    X_test,  y_test  = X.loc[idx_test],  y.loc[idx_test]
```

```
177 | else:
178 |     # First run: save the indices we just generated with the fixed random_state
179 |     np.save(f_train, X_train.index.values)
180 |     np.save(f_val,   X_val.index.values)
181 |     np.save(f_test,  X_test.index.values)
182 |
183 |
184 | # ====================================== L1 Logistic Regression ======================================
185 | pipe = Pipeline(steps=[
186 |     ("scaler", StandardScaler()),
187 |     ("clf", LogisticRegression(
188 |         penalty="l1",
189 |         solver="saga",
190 |         max_iter=5000,
191 |         class_weight="balanced",
192 |         random_state=SEED,
193 |         n_jobs=THREADS
194 |     ))
195 | ])
196 |
197 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
198 |
199 | param_grid = {"clf__C": np.logspace(-3, 1, 16)}
200 |
201 | grid = GridSearchCV(
202 |     estimator=pipe,
203 |     param_grid=param_grid,
204 |     scoring="roc_auc",
205 |     cv=cv,
206 |     n_jobs=1,
207 |     verbose=1
208 | )
209 |
210 | grid.fit(X_train, y_train)
211 | best_model = grid.best_estimator_
212 | print("Best C:", grid.best_params_["clf__C"])
213 | print("CV ROC-AUC (mean):", grid.best_score_)
214 |
215 | # Validate on VAL set
216 | val_proba = best_model.predict_proba(X_val)[:, 1]
217 | val_auc   = roc_auc_score(y_val, val_proba)
218 | val_prauc = average_precision_score(y_val, val_proba)
219 |
220 | # KS statistic
```

```
fpr, tpr, thr = roc_curve(y_val, val_proba)
ks_vals = tpr - fpr
ks = np.max(ks_vals)
thr_ks = thr[np.argmax(ks_vals)]
|
print(f"Validation ROC-AUC: {val_auc:.4f}")
print(f"Validation PR-AUC : {val_prauc:.4f}")
print(f"Validation KS     : {ks:.4f} @ threshold {thr_ks:.4f}")
|
# Choose threshold by max KS on validation
val_pred = (val_proba >= thr_ks).astype(int)
print("\nValidation Confusion Matrix:")
print(confusion_matrix(y_val, val_pred))
print("\nValidation Classification Report:")
print(classification_report(y_val, val_pred, digits=4))
|
# Retrain on TRAIN+VAL with best hyperparams, then evaluate on TEST
best_model_final = GridSearchCV(
    estimator=pipe,
    param_grid={"clf__C": [grid.best_params_["clf__C"]]},
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1
).fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val])).best_estimator_
|
test_proba = best_model_final.predict_proba(X_test)[:, 1]
test_auc   = roc_auc_score(y_test, test_proba)
test_prauc = average_precision_score(y_test, test_proba)
fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
ks_t = np.max(tpr_t - fpr_t)
|
# Use the threshold chosen on validation
test_pred = (test_proba >= thr_ks).astype(int)
|
print("\n=== TEST RESULTS (using validation KS threshold) ===")
print(f"Test ROC-AUC: {test_auc:.4f}")
print(f"Test PR-AUC : {test_prauc:.4f}")
print(f"Test KS     : {ks_t:.4f}")
print("\nTest Confusion Matrix:")
print(confusion_matrix(y_test, test_pred))
print("\nTest Classification Report:")
print(classification_report(y_test, test_pred, digits=4))
|
|
```

```
# ======================================== XGBoost ========================================
X_train = X_train.astype(np.float32, copy=False)
X_val   = X_val.astype(np.float32,   copy=False)
X_test  = X_test.astype(np.float32,  copy=False)

# CV setup
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
pos, neg = int((y_train == 1).sum()), int((y_train == 0).sum())
scale_pos_weight = neg / max(pos, 1)

pipe_xgb = Pipeline([
    ("imp", SimpleImputer(strategy="median")),
    ("xgb", XGBClassifier(
        objective="binary:logistic",
        eval_metric="auc",
        tree_method="hist",
        learning_rate=0.05,
        n_estimators=600,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_weight=1.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        scale_pos_weight=scale_pos_weight,
        n_jobs=THREADS,
        random_state=SEED,
        verbosity=0
    ))
])

# compact grid
param_grid_xgb = {
    "xgb__n_estimators": [400, 800],
    "xgb__max_depth": [4, 6],
    "xgb__learning_rate": [0.05, 0.1],
    "xgb__min_child_weight": [1.0, 3.0],
}

grid_xgb = GridSearchCV(
    estimator=pipe_xgb,
    param_grid=param_grid_xgb,
    scoring="roc_auc",
    cv=cv,
```

```
    n_jobs=1,
    verbose=1,
    refit=True
)

# Fit CV
grid_xgb.fit(X_train, y_train)
best_xgb = grid_xgb.best_estimator_
print("Best params:", grid_xgb.best_params_)
print("CV ROC-AUC :", grid_xgb.best_score_)
gc.collect()

# Validate on VAL — get KS threshold
val_proba = best_xgb.predict_proba(X_val)[:, 1]
val_auc   = roc_auc_score(y_val, val_proba)
val_prauc = average_precision_score(y_val, val_proba)
fpr, tpr, thr = roc_curve(y_val, val_proba)
ks_vals = tpr - fpr
ks = float(np.max(ks_vals))
thr_ks = float(thr[np.argmax(ks_vals)])

print(f"Validation ROC-AUC: {val_auc:.4f}")
print(f"Validation PR-AUC : {val_prauc:.4f}")
print(f"Validation KS     : {ks:.4f} @ threshold {thr_ks:.4f}")

val_pred = (val_proba >= thr_ks).astype(int)
print("\nValidation Confusion Matrix:")
print(confusion_matrix(y_val, val_pred))
print("\nValidation Classification Report:")
print(classification_report(y_val, val_pred, digits=4))
del val_proba, fpr, tpr, thr, ks_vals
gc.collect()

# Refit on TRAIN+VAL with best hyperparams
best_xgb_final = grid_xgb.best_estimator_
X_trv = pd.concat([X_train, X_val], copy=False)
y_trv = pd.concat([y_train, y_val], copy=False)
best_xgb_final.fit(X_trv, y_trv)
del X_trv, y_trv
gc.collect()

# Final TEST evaluation (use validation KS threshold)
test_proba = best_xgb_final.predict_proba(X_test)[:, 1]
test_auc   = roc_auc_score(y_test, test_proba)
```

```
test_prauc = average_precision_score(y_test, test_proba)
fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
ks_t = float(np.max(tpr_t - fpr_t))
test_pred = (test_proba >= thr_ks).astype(int)

print("\n=== TEST RESULTS (using validation KS threshold) ===")
print(f"Test ROC-AUC: {test_auc:.4f}")
print(f"Test PR-AUC : {test_prauc:.4f}")
print(f"Test KS     : {ks_t:.4f}")
print("\nTest Confusion Matrix:")
print(confusion_matrix(y_test, test_pred))
print("\nTest Classification Report:")
print(classification_report(y_test, test_pred, digits=4))


# ======================================== CatBoost ========================================
X_train_cb = X_train.copy()
X_val_cb   = X_val.copy()
X_test_cb  = X_test.copy()

# Class weight for imbalance
pos = int((y_train == 1).sum())
neg = int((y_train == 0).sum())
class_weights = [1.0, neg / max(pos, 1)]

# Small param grid
param_grid_cb = [
    {"depth": 6,  "learning_rate": 0.05, "l2_leaf_reg": 3.0},
    {"depth": 8,  "learning_rate": 0.05, "l2_leaf_reg": 3.0},
    {"depth": 6,  "learning_rate": 0.1,  "l2_leaf_reg": 3.0},
]

best_cb = None
best_auc = -np.inf
best_params = None

# Simple manual CV loop for small grid
for params in param_grid_cb:
    cb = CatBoostClassifier(
        task_type="CPU",
        iterations=2000,
        early_stopping_rounds=100,
        loss_function="Logloss",
        eval_metric="AUC",
```

```
            class_weights=class_weights,
            random_seed=SEED,
            thread_count=THREADS,
            verbose=False,
            **params
        )
        cb.fit(X_train_cb, y_train, eval_set=(X_val_cb, y_val), use_best_model=True)
        val_proba = cb.predict_proba(X_val_cb)[:, 1]
        val_auc = roc_auc_score(y_val, val_proba)
        if val_auc > best_auc:
            best_auc = val_auc
            best_cb = cb
            best_params = params

print("Best CatBoost params:", best_params)
print("Validation ROC-AUC:", best_auc)

# Compute validation PR-AUC and KS
val_proba = best_cb.predict_proba(X_val_cb)[:, 1]
val_prauc = average_precision_score(y_val, val_proba)
fpr, tpr, thr = roc_curve(y_val, val_proba)
ks_vals = tpr - fpr
ks = float(np.max(ks_vals))
thr_ks_cb = float(thr[np.argmax(ks_vals)])
print(f"Validation PR-AUC: {val_prauc:.4f}")
print(f"Validation KS    : {ks:.4f} @ threshold {thr_ks_cb:.4f}")

val_pred = (val_proba >= thr_ks_cb).astype(int)
print("\nValidation Confusion Matrix:")
print(confusion_matrix(y_val, val_pred))
print("\nValidation Classification Report:")
print(classification_report(y_val, val_pred, digits=4))

# Retrain on TRAIN+VAL
cb_final = CatBoostClassifier(
    task_type="CPU",
    iterations=2000,
    early_stopping_rounds=100,
    loss_function="Logloss",
    eval_metric="AUC",
    class_weights=class_weights,
    random_seed=SEED,
    verbose=False,
    **best_params
```

```
441 | )
442 | X_trv_cb = pd.concat([X_train_cb, X_val_cb], copy=False)
443 | y_trv_cb = pd.concat([y_train, y_val], copy=False)
444 | cb_final.fit(X_trv_cb, y_trv_cb, use_best_model=False)
445 |
446 | # Test set evaluation
447 | test_proba = cb_final.predict_proba(X_test_cb)[:, 1]
448 | test_auc   = roc_auc_score(y_test, test_proba)
449 | test_prauc = average_precision_score(y_test, test_proba)
450 | fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
451 | ks_t = float(np.max(tpr_t - fpr_t))
452 | test_pred = (test_proba >= thr_ks_cb).astype(int)
453 |
454 | print("\n=== TEST RESULTS (CatBoost, using validation KS threshold) ===")
455 | print(f"Test ROC-AUC: {test_auc:.4f}")
456 | print(f"Test PR-AUC : {test_prauc:.4f}")
457 | print(f"Test KS     : {ks_t:.4f}")
458 | print("\nTest Confusion Matrix:")
459 | print(confusion_matrix(y_test, test_pred))
460 | print("\nTest Classification Report:")
461 | print(classification_report(y_test, test_pred, digits=4))
462 | gc.collect()
463 |
464 |
465 | # ======================================== Random Forest ========================================
466 | X_train_rf = X_train.astype(np.float32, copy=False)
467 | X_val_rf   = X_val.astype(np.float32,   copy=False)
468 | X_test_rf  = X_test.astype(np.float32,  copy=False)
469 |
470 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
471 |
472 | pipe_rf = Pipeline([
473 |     ("imp", SimpleImputer(strategy="median")),
474 |     ("rf", RandomForestClassifier(
475 |         n_estimators=600,
476 |         max_depth=None,
477 |         max_features="sqrt",
478 |         min_samples_split=2,
479 |         min_samples_leaf=1,
480 |         class_weight="balanced_subsample",
481 |         n_jobs=max(1, os.cpu_count() // 2),
482 |         random_state=SEED
483 |     ))
484 | ])
```

```
485 |
486 | # compact grid
487 | param_grid_rf = {
488 |     "rf__n_estimators": [400, 800],
489 |     "rf__max_depth": [None, 12],
490 |     "rf__min_samples_leaf": [1, 5],
491 |     "rf__max_features": ["sqrt", 0.5],
492 | }
493 |
494 | grid_rf = GridSearchCV(
495 |     estimator=pipe_rf,
496 |     param_grid=param_grid_rf,
497 |     scoring="roc_auc",
498 |     cv=cv,
499 |     n_jobs=1,
500 |     verbose=1,
501 |     refit=True
502 | )
503 |
504 | # Fit CV
505 | grid_rf.fit(X_train_rf, y_train)
506 | best_rf = grid_rf.best_estimator_
507 | print("Best RF params:", grid_rf.best_params_)
508 | print("CV ROC-AUC    :", grid_rf.best_score_)
509 |
510 | # Validate — pick KS threshold
511 | val_proba = best_rf.predict_proba(X_val_rf)[:, 1]
512 | val_auc   = roc_auc_score(y_val, val_proba)
513 | val_prauc = average_precision_score(y_val, val_proba)
514 | fpr, tpr, thr = roc_curve(y_val, val_proba)
515 | ks_vals = tpr - fpr
516 | ks = float(np.max(ks_vals))
517 | thr_ks_rf = float(thr[np.argmax(ks_vals)])
518 |
519 | print(f"Validation ROC-AUC: {val_auc:.4f}")
520 | print(f"Validation PR-AUC : {val_prauc:.4f}")
521 | print(f"Validation KS     : {ks:.4f} @ threshold {thr_ks_rf:.4f}")
522 |
523 | val_pred = (val_proba >= thr_ks_rf).astype(int)
524 | print("\nValidation Confusion Matrix:")
525 | print(confusion_matrix(y_val, val_pred))
526 | print("\nValidation Classification Report:")
527 | print(classification_report(y_val, val_pred, digits=4))
528 | del val_proba, fpr, tpr, thr, ks_vals
```

```
gc.collect()

# Refit on TRAIN+VAL with best hyperparams
best_rf_final = grid_rf.best_estimator_
X_trv_rf = pd.concat([X_train_rf, X_val_rf], copy=False)
y_trv_rf = pd.concat([y_train, y_val],       copy=False)
best_rf_final.fit(X_trv_rf, y_trv_rf)
del X_trv_rf, y_trv_rf
gc.collect()

# Final TEST evaluation (use validation KS threshold)
test_proba = best_rf_final.predict_proba(X_test_rf)[:, 1]
test_auc   = roc_auc_score(y_test, test_proba)
test_prauc = average_precision_score(y_test, test_proba)
fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
ks_t = float(np.max(tpr_t - fpr_t))
test_pred = (test_proba >= thr_ks_rf).astype(int)

print("\n=== TEST RESULTS (Random Forest, using validation KS threshold) ===")
print(f"Test ROC-AUC: {test_auc:.4f}")
print(f"Test PR-AUC : {test_prauc:.4f}")
print(f"Test KS     : {ks_t:.4f}")
print("\nTest Confusion Matrix:")
print(confusion_matrix(y_test, test_pred))
print("\nTest Classification Report:")
print(classification_report(y_test, test_pred, digits=4))


# ======================================== PLOTS: ROC / PR / KS on TEST for 4 models (Logit L1, XGB, CatBoost, RF) ===============
# Collect test-set probabilities
probas = {
    "L1-Logit":     best_model_final.predict_proba(X_test)[:, 1],
    "XGBoost":      best_xgb_final.predict_proba(X_test)[:, 1],
    "CatBoost":     cb_final.predict_proba(X_test_cb)[:, 1],
    "RandomForest": best_rf_final.predict_proba(X_test_rf)[:, 1],
}

# Summary table: AUC / PR-AUC / KS (max TPR-FPR)
summary_rows = []
ks_curves = {}
for name, p in probas.items():
    fpr, tpr, thr = roc_curve(y_test, p)
    ks_vals = tpr - fpr
    ks = float(np.max(ks_vals))
```

```
    thr_ks = float(thr[np.argmax(ks_vals)])
    auc = float(roc_auc_score(y_test, p))
    pr_auc = float(average_precision_score(y_test, p))
    summary_rows.append([name, auc, pr_auc, ks, thr_ks])
    ks_curves[name] = (thr, ks_vals)

summary = pd.DataFrame(summary_rows, columns=["Model", "ROC-AUC", "PR-AUC", "KS", "KS_threshold"]) \
            .sort_values("ROC-AUC", ascending=False)
print("\n=== TEST summary ===")
print(summary.to_string(index=False))

# ROC curve
plt.figure()
for name, p in probas.items():
    fpr, tpr, _ = roc_curve(y_test, p)
    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test, p):.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves on TEST")
plt.legend()
plt.tight_layout()
plt.savefig("roc_test.png", dpi=200)
plt.show()

# PR curve
plt.figure()
for name, p in probas.items():
    prec, rec, _ = precision_recall_curve(y_test, p)
    plt.plot(rec, prec, label=f"{name} (PR-AUC={average_precision_score(y_test, p):.3f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision–Recall Curves on TEST")
plt.legend()
plt.tight_layout()
plt.savefig("pr_test.png", dpi=200)
plt.show()

# KS curve
plt.figure()
for name, (thr, ks_vals) in ks_curves.items():
    plt.plot(thr, ks_vals, label=f"{name}")
plt.xlabel("Threshold")
plt.ylabel("KS = TPR - FPR")
```

```
617 | plt.title("KS Curves on TEST")
618 | plt.legend()
619 | plt.tight_layout()
620 | plt.savefig("ks_test.png", dpi=200)
621 | plt.show()
622 |
623 | # ======================================== Portfolio VaR & ES on TEST (Monte Carlo, independent defaults) ========================
624 |
625 | ALPHAS = [0.95, 0.975, 0.99]
626 | NSIMS  = 8000
627 |
628 | N_test = len(y_test)
629 | EAD_VEC    = np.ones(N_test, dtype=float)
630 | LGD_SCALAR = 1.0
631 | W = EAD_VEC * LGD_SCALAR
632 |
633 | def simulate_portfolio_losses(p_vec, w_vec, nsims=NSIMS, seed=SEED):
634 |     """Simulate total portfolio loss nsims times for Bernoulli defaults with probs p_vec."""
635 |     rng = np.random.default_rng(seed)
636 |     n   = p_vec.shape[0]
637 |     out = np.empty(nsims, dtype=float)
638 |     for s in range(nsims):
639 |         defaults = rng.random(n) < p_vec
640 |         out[s] = float(np.dot(defaults, w_vec))
641 |     return out
642 |
643 | def var_es_from_losses(losses, alpha):
644 |     """Empirical VaR and ES from a vector of simulated losses."""
645 |     var = float(np.quantile(losses, alpha, method="higher"))
646 |     es  = float(losses[losses >= var].mean())
647 |     return var, es
648 |
649 | rows = []
650 |
651 | def pct_label(a: float) -> str:
652 |     x = a * 100
653 |     return f"{x:.1f}".rstrip("0").rstrip(".")
654 |
655 | for name, p in probas.items():
656 |     p = np.asarray(p, dtype=float)
657 |     seed_model = SEED + int.from_bytes(
658 |         hashlib.sha256(f"{SEED}:{name}".encode("utf-8")).digest()[:4],
659 |         "little"
660 |     )
```

```
661 |
662 |     losses = simulate_portfolio_losses(p, W, nsims=NSIMS, seed=seed_model)
663 |     meanL  = float(losses.mean())
664 |     stdL   = float(losses.std(ddof=1))
665 |     row = {
666 |         "Model": name,
667 |         "MeanLoss": meanL,
668 |         "StdLoss": stdL,
669 |         "MeanLossRate": meanL / np.sum(EAD_VEC)
670 |     }
671 |     for a in ALPHAS:
672 |         v, e = var_es_from_losses(losses, a)
673 |         row[f"VaR@{pct_label(a)}"]     = v
674 |         row[f"ES@{pct_label(a)}"]      = e
675 |         row[f"VaRrate@{pct_label(a)}"] = v / np.sum(EAD_VEC)
676 |         row[f"ESrate@{pct_label(a)}"]  = e / np.sum(EAD_VEC)
677 |     rows.append(row)
678 |
679 | var_es_df = pd.DataFrame(rows).sort_values(f"VaR@{pct_label(ALPHAS[-1])}", ascending=False)
680 |
681 | print("\n=== TEST Portfolio VaR & ES (counts and rates; assumes indep. defaults, EAD=1, LGD=1) ===")
682 | cols_order = ["Model", "MeanLoss", "StdLoss",
683 |               *(f for a in ALPHAS for f in (f"VaR@{pct_label(a)}", f"ES@{pct_label(a)}")),
684 |               "MeanLossRate",
685 |               *(f for a in ALPHAS for f in (f"VaRrate@{pct_label(a)}", f"ESrate@{pct_label(a)}"))]
686 | print(var_es_df[cols_order].to_string(index=False))
687 |
688 |
689 | # ============================ Export terminal output to pdf ============================
690 | try:
691 |     from fpdf import FPDF
692 |     pdf = FPDF()
693 |     pdf.set_auto_page_break(auto=True, margin=10)
694 |     pdf.add_page()
695 |     pdf.set_font("Courier", size=9)
696 |     with open("run_output.txt", encoding="utf-8") as f:
697 |         for line in f:
698 |             pdf.multi_cell(0, 4, line.rstrip("\n"))  # wraps long lines
699 |     pdf.output("run_output.pdf")
700 |     print("\nSaved PDF: run_output.pdf")
701 | except Exception as e:
702 |     print(f"\nPDF export skipped: {e}\n(Install with: pip install fpdf2)")
```